Operating Systems Assignment 2 – Task 4 (Q4)

Inter-Process Communication – Weather Station Data Pipeline (CO2, K5 – Evaluate, 6 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs. Compile: `gcc -std=c11 -O2 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

1. Model

A **sensor process** (producer) reads sensors and must pass a continuous stream of readings to an **analysis process** (consumer) without losing any. The two processes have separate address spaces, so they need an IPC mechanism. Two methods are implemented: (A) a pipe, (B) shared memory protected by semaphores.

2. Concepts to determine

Question	Answer
How does a pipe signal end of data?	When the writer closes **all** copies of the write end, the reader's `read()` returns **0** (EOF) after the remaining data is drained. The reader therefore loops until read() returns 0. If any process still holds the write end open, read() blocks instead.
Shared memory vs message passing	**Message passing** (pipe, message queue, socket): data is sent through the kernel with send/receive operations; the kernel handles synchronisation and buffering; simple and safe, slower for large data. **Shared memory**: both processes map the same memory region and read/write it directly; the kernel is involved only in setting it up, so it is fastest, but the programmer must provide synchronisation.
Which method copies the data?	**Pipe copies**: user buffer → kernel pipe buffer (copy 1), kernel buffer → reader's buffer (copy 2). **Shared memory does not copy through the kernel**: the consumer reads the same physical pages the producer wrote (the producer still writes the data into the region once).
Which needs extra synchronisation, and why?	**Shared memory**. Nothing stops the producer overwriting a slot the consumer has not read, or the consumer reading a slot not yet written; this is a race condition / bounded-buffer problem, solved here with two semaphores (`empty`, `full`). A pipe has the kernel's blocking behaviour built in: write() blocks when the pipe is full and read() blocks when it is empty.

3. Implement A: pipe – Complete the Code (TODO 1)

TODO 1 is closing the unused pipe ends: the child closes `fd[1]` (it only reads) and the parent closes `fd[0]` (it only writes). Then the parent closes `fd[1]` after sending the last reading so the child gets EOF.

```
/* Task 4 - Weather station: sensor process -&gt; analysis process through a pipe.
   TODO 1 (completed): close the unused pipe ends in each process.
   gcc -std=c11 -Wall -Wextra -o pipe_demo pipe_demo.c
   ./pipe_demo          normal run
   ./pipe_demo noclose  parent does NOT close fd[1]  -&gt; child never sees EOF */
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;sys/wait.h&gt;
typedef struct { int id; double temp, humidity; } Reading;
int main(int argc, char *argv[]) {
    int noclose = (argc &gt; 1 &amp;&amp; strcmp(argv[1], "noclose") == 0);
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid &lt; 0) { perror("fork"); return 1; }
    if (pid == 0) {                                  /* analysis process (reader) */
        close(fd[1]);                                /* TODO 1: child closes write end */
        Reading r; int n = 0; double sum = 0; ssize_t k;
        while ((k = read(fd[0], &amp;r, sizeof r)) == (ssize_t)sizeof r) { n++; sum += r.temp; }
        /* read() returns 0 (EOF) only when EVERY write end is closed */
        printf("analysis: read() returned %zd (EOF) after %d readings, mean temp %.2f\n", k, n, n ? sum / n : 0);
        close(fd[0]);
        return 0;
    }
    close(fd[0]);                                    /* TODO 1: parent closes read end */
    for (int i = 1; i &lt;= 10; i++) {                   /* sensor process (writer) */
        Reading r = {i, 20.0 + i * 0.5, 60.0 - i};
        write(fd[1], &amp;r, sizeof r);
    }
    if (!noclose) close(fd[1]);                      /* signals end of data to the reader */
```

```
    else printf("sensor: NOT closing write end, waiting for child...\n");
    fflush(stdout);
    wait(NULL);
    printf("sensor: child finished\n");
    return 0;
}
```

Execution – normal run

```
analysis: read() returned 0 (EOF) after 10 readings, mean temp 22.75
sensor: child finished
```

All 10 readings arrive and the analysis process sees read() = 0 (EOF) and terminates.

Expected-output experiment: remove close(fd[1]) in the parent

```
sensor: NOT closing write end, waiting for child...
exit status: 124 (124 = killed by timeout, child blocked in read())
```

Why the child never finishes: a pipe reports EOF only when *no* process has the write end open. The parent still holds `fd[1]` open (it is blocked in `wait()`, so it can never close it later), so the kernel cannot tell that no more data will come. The child's `read()` therefore blocks forever waiting for data, and the parent's `wait()` waits for the child: a **deadlock**. The run above had to be killed by `timeout` after 3 s (exit status 124). Similarly, if the child did not close its own copy of `fd[1]`, it would also keep the pipe open and never see EOF.

4. Implement B: shared memory with semaphores

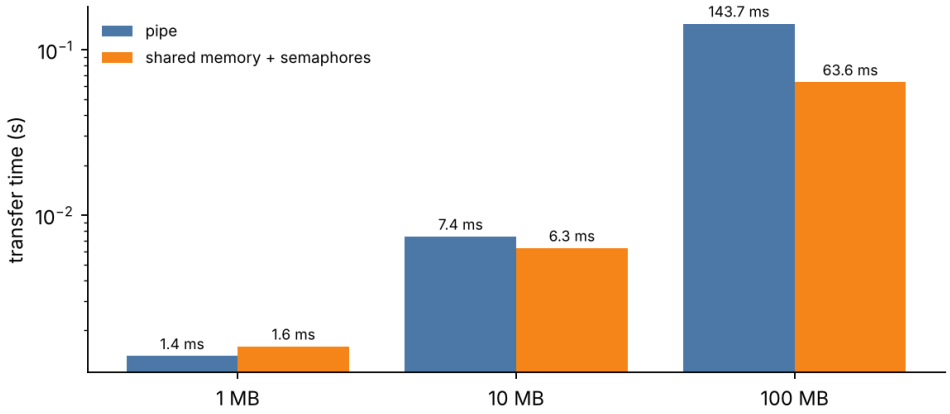
`ipc.c` sends N MB from a producer process to a consumer process using either a pipe (64 KB chunks) or a shared-memory ring buffer of 8 slots × 64 KB created with `mmap(MAP_SHARED|MAP_ANONYMOUS)`. Two process-shared POSIX semaphores synchronise the slots: `empty` counts free slots (initially 8), `full` counts filled slots (initially 0). A slot length of 0 marks end of data. The consumer calculates a checksum, so lost or corrupted data would be detected.

```
/* Transfer N MB from a producer process to a consumer process.
   pipe : kernel pipe, 64 KB chunks              (copy user-&gt;kernel-&gt;user)
   shm  : shared ring buffer + POSIX semaphores  (no kernel copy of the payload)
   gcc -std=c11 -O2 -Wall -Wextra -pthread -o ipc ipc.c      ./ipc &lt;pipe|shm&gt; &lt;MB&gt; */
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;semaphore.h&gt;
#include &lt;sys/mman.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;time.h&gt;
#define CH 65536
#define SLOTS 8
typedef struct { sem_t empty, full; size_t len[SLOTS]; char buf[SLOTS][CH]; } Ring;
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&amp;t);return t.tv_sec+t.tv_nsec*1e-9;}
static unsigned long sum(const char*p,size_t n){unsigned long s=0;for(size_t i=0;i&lt;n;i++)s+=(unsigned char)p[i];
return s;}
int main(int argc,char**argv){
    const char*mode=argc&gt;1?argv[1]:"pipe"; size_t total=(size_t)(argc&gt;2?atoi(argv[2]):1)&lt;&lt;20;
    static char src[CH]; for(size_t i=0;i&lt;CH;i++) src[i]=(char)(i*7);
    unsigned long expect=0; for(size_t o=0;o&lt;total;o+=CH) expect+=sum(src,CH);
    double t0=now();
    if(!strcmp(mode,"pipe")){
        int fd[2]; pipe(fd);
        if(fork()==0){ close(fd[1]); static char b[CH]; unsigned long s=0; size_t got=0; ssize_t k;
            while((k=read(fd[0],b,CH))&gt;0){ s+=sum(b,k); got+=k; }
            printf("consumer: %zu bytes, checksum %s\n",got,s==expect?"OK":"MISMATCH"); fflush(stdout); _exit(0);}
        close(fd[0]); for(size_t o=0;o&lt;total;o+=CH) write(fd[1],src,CH);
        close(fd[1]); wait(NULL);
    } else {
        Ring*r=mmap(NULL,sizeof(Ring),PROT_READ|PROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-1,0);
        sem_init(&amp;r-&gt;empty,1,SLOTS); sem_init(&amp;r-&gt;full,1,0);          /* pshared = 1 */
        if(fork()==0){ unsigned long s=0; size_t got=0; int i=0;
            for(;;){ sem_wait(&amp;r-&gt;full); size_t n=r-&gt;len[i]; if(n==0) break;   /* len 0 = end of data */
                s+=sum(r-&gt;buf[i],n); got+=n; sem_post(&amp;r-&gt;empty); i=(i+1)%SLOTS; }
            printf("consumer: %zu bytes, checksum %s\n",got,s==expect?"OK":"MISMATCH"); fflush(stdout); _exit(0);}
        int i=0; for(size_t o=0;o&lt;total;o+=CH){ sem_wait(&amp;r-&gt;empty); memcpy(r-&gt;buf[i],src,CH); r-&gt;
len[i]=CH; sem_post(&amp;r-&gt;full); i=(i+1)%SLOTS; }
        sem_wait(&amp;r-&gt;empty); r-&gt;len[i]=0; sem_post(&amp;r-&gt;full); wait(NULL);
    }
    printf("%-4s %4zu MB  %.4f s  %.0f MB/s\n",mode,total&gt;&gt;20,now()-t0,(total&gt;&gt;20)/(now()-t0));
    return 0; }
```

5. Measure: transfer of 1 MB, 10 MB and 100 MB

Median of 5 runs; both methods include the same consumer checksum work, and the checksum was **OK for every run** (no data lost). Script: `bench.sh`.

Size	Pipe (s)	Pipe MB/s	Shared memory + semaphores (s)	Shm MB/s	Shm faster by
1 MB	0.0014	714	0.0016	625	0.88×
10 MB	0.0074	1351	0.0063	1587	1.17×
100 MB	0.1437	696	0.0636	1572	2.26×



6. Evaluate and compare

Growth: time grows with size and is O(n) in theory. Measured, small runs are dominated by fixed costs, so growth is not exactly 10× per step: from 10 MB to 100 MB the pipe time rises about 19× (0.0074 → 0.144 s, ≈ 700 MB/s) and shared memory about 10× (0.0063 → 0.064 s, ≈ 1,570 MB/s). The pipe's extra slowdown at 100 MB is consistent with its two copies per byte, though I did not profile the cause. At **1 MB and 10 MB the difference is small** (within about 1.2× and within run-to-run noise): fixed costs (process creation, setup, scheduling) dominate such short runs. At **100 MB shared memory is about 2.3× faster**, since it avoids the two kernel copies that the pipe makes for every byte, so its advantage grows with data size.

Why shared memory is not infinitely faster: the producer still writes each byte once (memcpy into the ring), the consumer still reads each byte once (checksum), and every 64 KB slot costs two semaphore operations (sem_wait/sem_post, potentially a futex system call when blocking). A pipe needs one write() and one read() system call per chunk plus the two copies.

Criterion	Pipe	Shared memory + semaphores
Model	Message passing (byte stream, kernel buffer)	Shared memory
Copies of the data	2 through the kernel per transfer	0 kernel copies (data stays in the shared pages)
Extra synchronisation	None (kernel blocks readers/writers)	Required: semaphores/mutex (race risk if wrong)
End of data	Writer closes → read() returns 0	Must be defined by the programmer (here slot length 0)
Ease and safety	Simple; hard to misuse	Complex; deadlock/race bugs possible
Speed on 100 MB (measured)	0.144 s	0.064 s
Works between unrelated processes	Anonymous pipe: only related processes (named FIFO for unrelated)	Yes, with named shm (shm_open) or inherited mapping

7. Alternative concept: shared memory + semaphore instead of a pipe – complexity

Solution (bounded buffer, as in ipc.c):

```
producer:  for each reading r:  sem_wait(empty); buf[in] = r; in = (in+1) % K; sem_post(full)
```

```
consumer:  for each reading:     sem_wait(full);  r = buf[out]; out = (out+1) % K; sem_post(empty)
init:      empty = K, full = 0; ring lives in shared memory (1 producer + 1 consumer: no mutex needed)
```

For n readings of size s	Pipe	Shared memory + semaphore
Time	O(n·s) data copying: 2 copies (user→kernel→user) = 2n·s bytes moved by the kernel, plus ≈ n·s/chunk system calls	O(n·s) at most one write by producer and one read by consumer, no kernel copy; ≈ 2 semaphore ops per slot (system calls only when blocking)
Space	O(pipe buffer) = 64 KB by default in the kernel + user buffers	O(K·slot size) shared region, fixed and chosen by the programmer
Synchronisation	Implicit in read()/write() blocking	Explicit semaphores (empty/full) – extra code and risk

Both are linear in n; the shared-memory solution has a smaller constant (no kernel copying), which shows up clearly at 100 MB (2.3×). For a single weather station sending a few bytes per second, the difference is invisible.

8. Design decision

Choose a pipe (or a message queue) for the weather station. Sensor readings are small and arrive slowly (a few hundred bytes per second), so the copy overhead is irrelevant, whereas a pipe gives built-in blocking synchronisation, no lost readings and a clear end-of-data signal with no extra code or locking bugs. **Use shared memory with semaphores only if the data rate becomes large** (for example images or raw high-frequency waveforms of tens to hundreds of MB), where the measured 2.3× advantage is worth the extra complexity. In both cases the unused pipe ends must be closed, otherwise the analysis process never receives end of data.

Limitations

Timings come from a 2-core virtualised container, so absolute speeds will differ on your machine; small-size runs (1 MB, 10 MB) are noisy. I implemented the pipe and shared-memory methods; a POSIX message queue was not implemented since the task allows either one. Run the programs yourself, commit `Task04/pipe_demo.c, ipc.c, bench.sh` and outputs, and add your name, register number and GitHub link.